THE CIRCLE: SECURITY & PENETRATION TEST REPORT

Assessment Date: May 2026 | Standard Compliance: OWASP Top 10

This report compiles security findings, exploitation vectors, and mitigation details verified on The CirCle during active code review and vulnerability testing cycles.

---

1. Executive Summary

The CirCle undergoes rigorous security checks to guarantee user data privacy, multi-tenant isolation, and resource protection. During development, three potential security vectors were identified, simulated, and resolved. Mitigations have been verified against OWASP Top 10 standards.

---

2. Assessment Details

- **Target:** The CirCle REST API & Authentication Layer
- **Scope:** JWT Sessions, Workspace Membership Controls, Tenant Analytics
- **Testing Style:** White-box source code review & simulated request manipulation

---

3. Vulnerability Logs & Mitigations

3.1 Vulnerability 1: Change Password Authentication Bypass (Broken Authentication)

- **Risk Rating:** High (CVSS v3: 8.1)
- **Description:**

The password update endpoint (`/api/auth/change-password`) originally accepted password changes by verifying only the signature of the JWT token. It did not require the user to provide their current active password. Under this configuration, if an attacker obtained an active access token or hijacked a browser session (e.g., via temporary physical access), they could change the user's password instantly, locking the legitimate user out indefinitely.

- **Exploitation Path:**

1. Attacker copies JWT access token from cookies or local storage.
2. Attacker crafts a POST request to `/api/auth/change-password` supplying only `{ "newPassword": "attacker_password" }`.
3. The server processes the request and updates the hash in MongoDB.

- **Mitigation Implementation:**

We updated the authentication router logic. The password change route now enforces verification of the current password:

```
// server/routes/auth.ts
const { currentPassword, newPassword } = req.body;
const user = await UserModel.findById(req.user.id);
const isValid = await bcrypt.compare(currentPassword, user.password);
if (!isValid) {
  return res.status(401).json({ error: "Invalid current password" });
```

```
    }
    user.password = await bcrypt.hash(newPassword, 12);
    await user.save();
```

This ensures that a hijacked session cannot change credentials without knowing the original password.

3.2 Vulnerability 2: BOLA on Workspace Memberships (Broken Object Level Authorization)

- **Risk Rating:** Critical (CVSS v3: 9.3)
- **Description:**

A member of one workspace could modify membership and change user roles of another workspace by directly calling the update endpoint `/api/workspaces/:id/members` and passing a target Workspace ID. The endpoint verified that the user was authenticated, but did not assert that the user possessed the `Owner` or `Admin` role *within that specific workspace context*.

- **Exploitation Path:**

1. User A (who is a low-privilege Member in Workspace 1) obtains the ObjectId of Workspace 2.
2. User A sends a PUT request to `/api/workspaces/Workspace2-ObjectId/members` with payload `{ "userId": "UserA-ID", "role": "Owner" }`.
3. Server grants Owner rights to User A on Workspace 2.

- **Mitigation Implementation:**

We introduced a workspace permission gating middleware (`server/middleware/permission.ts`) that resolves the user's workspace membership and role dynamically. Workspace modification routes now require explicit role validation:

```
    // server/routes/workspaces.ts
    router.put("/:id/members", async (req, res) => {
      const role = await getWorkspaceRole(req.user.id, req.params.id, req.user.role,
req.user.organisationId);
      if (role !== "Owner" && role !== "Admin") {
        return res.status(403).json({ error: "Forbidden: Only Owners or Admins can edit
memberships" });
      }
      // Proceed with membership update...
    });
```

This prevents cross-workspace BOLA privilege escalation.

3.3 Vulnerability 3: Unauthenticated Dashboard Statistics Leak (Sensitive Data Exposure)

- **Risk Rating:** Medium (CVSS v3: 5.3)
- **Description:**

The server served tenant stats (counts of projects, epics, tasks, and members) via `/api/dashboard/stats` to any HTTP request. The endpoint did not attach authentication middleware, allowing malicious crawlers to scrape metrics, project volumes, and user counts of any tenant.

- **Exploitation Path:**

1. Attacker sends GET request to `http://localhost:3000/api/dashboard/stats?organisationId=Org-ObjectId`.
2. Server responds with the database metrics of that organization.

- **Mitigation Implementation:**

We bound the JWT verification middleware to the dashboard router and restricted query results to the user's active session organization context:

```
    // server/routes/dashboard.ts
    router.get("/stats", authenticateJWT, async (req, res) => {
      const organisationId = req.user.organisationId;
      const projectCount = await ProjectModel.countDocuments({ organisationId });
      const taskCount = await WorkItemModel.countDocuments({ organisationId });
      res.json({ projectCount, taskCount });
    });
```

This guarantees that dashboard analytics are shielded from unauthenticated access.

---

4. Conclusion

All identified vulnerabilities have been mitigated and verified through automated unit tests and API checks. The CirCle satisfies secure multi-tenant isolation, credential protection, and API data governance boundaries.